

Ejercicio 1

Inciso (a)

Es importante escalar los valores de entrada porque:

1. Acelera la convergencia del entrenamiento. Si las features tienen escalas muy distintas, los gradientes avanzan más hacia algunas direcciones (la de los features no escalados seguramente), haciendo que el descenso por gradiente zigzaguee y tarde en converger para encontrar el mínimo error.
2. Evita vanishing/exploding gradients. Mantener valores en rangos equiparados evita que las activaciones crezcan o se reduzcan mucho en backpropagation.
3. Mejora la estabilidad numérica. Sirve para ADAM y RMSProp.
4. Inicialización de pesos más fácil por tener distribuciones consistentes.

Inciso (b)

Es problemático porque implica inicializar todos los pesos en cero. Esto implica que todas las neuronas:

1. Reciben la misma información.
2. Generan la misma salida.
3. Reciben el mismo gradiente.
4. Se actualizan de la misma forma.

Las neuronas terminan siendo todas iguales y la red no aprende nada útil.

Inciso (c)

Sabemos que con dropout se apagan neuronas aleatoriamente por cada iteración en el entrenamiento. Esto va a generar un flujo menor de activaciones que terminarán afectando a la predicción. Como hay ciertas activaciones que se pierden, los pesos aprendidos se inflarán para compensar las neuronas que se apagaron, porque los pesos se ajustaron para trabajar con activaciones más pequeñas. Por eso, se corrige escalando las activaciones con **inverted dropout** para que la distribución de activaciones en test coincida con la de entrenamiento.

Inciso (d)

Cuando se entrena una red, se necesita calcular el gradiente de la función de pérdida para actualizar los pesos, pero no siempre se usa todo el dataset para calcularlo. Por eso se usan batch(es) para agrupar ciertas observaciones y así calcular la pérdida promedio de ese grupo y su gradiente.

Cuando el tamaño del batch es 1, es el gradiente descendente (SGD) tradicional, porque cada actualización de los pesos usa una sola observación. Si tengo 10.000 observaciones,

los pesos se pueden actualizar hasta 10.000 veces. Van a ser actualizaciones rápidas pero inestables, difícilmente converja de manera suave y sirven para escapar de mínimos locales.

Cuando el tamaño del batch es N , se calcula la pérdida promedio sobre todas las observaciones del dataset. Va a tener actualizaciones muy estables y un gradiente suave. Pero son muy lentas por su costo y se pueden quedar trabadas en mínimos locales.

Ejercicio 2

- **Batch Normalization** previene exploding gradient porque normaliza las activaciones. También previene vanishing gradient porque fuerza la varianza de las activaciones a $\sigma^2 \approx 1$ y su media a $\mu \approx 0$. Previene activaciones muy grandes y activaciones muy chicas para estabilizar el forwardpropagation y backpropagation.
- **Momentum** no soluciona el vanishing gradient. Simplemente porque es un optimizador que toma gradientes pasados para que converja más rápido, no es un normalizador de activaciones.
- **Estrategias de inicialización de pesos** sí ayuda al vanishing gradient porque evita que las activaciones resulten muy pequeñas. En backpropagation, el gradiente depende directamente de los pesos.
- **Aumentar las capas de la red neuronal** no ayuda al vanishing gradient, sino que lo empeora. Como el backpropagation consiste en multiplicar gradientes, pesos y biases, tener más capas implica que se llegue posiblemente a un valor muy pequeño o muy grande.
- **Utilizar LogSoftmax en vez de Softmax** no ayuda al vanishing gradient porque no toca la magnitud de las activaciones y/o gradientes, porque simplemente se usa en la capa de salida para estabilización numérica luego de usar Softmax.

Ejercicio 3

1. Imágenes: rotar, flip, modificar exposición a la luz, modificar contraste, zoom, recorte.
2. Audio: transponer a una octava más grave o más aguda, recortar, comprimir, cambiar volumen.
3. Video: mezclar cuadros, quitar cuadros, duplicar cuadros. Más lo de las imágenes.
4. Texto: errores tipográficos, traducción ida y vuelta, palabras irrelevantes, sinónimos.

Conviene aclarar que para hacer una augmentación tiene que preservar la etiqueta, no se debe distorsionar mucho el dato original y debe generar datos que puedan darse en la vida real.

Ejercicio 4

Inciso (a)

Momentum sirve para acelerar la convergencia en gradient descent, buscando suavizar los updates y pasar las llanuras/sillas de montar. La idea de la actualización de los pesos consiste en agregar una fracción del gradiente anterior al gradiente actual. Funciona como un parámetro de velocidad. Su fórmula es

$$v_t = \alpha v_{t-1} - \epsilon \Delta f(x_t)$$

Inciso (b)

Momentum no adapta el learning rate por parámetro, sino que usa uno global para todos los parámetros. Momentum no sabe si un parámetro necesita un paso grande o uno chico para ajustarse.

ADAGrad ajusta automáticamente el paso para cada parámetro según la magnitud histórica de sus gradientes (cuánto gradiente recibió a lo largo del tiempo), haciendo el entrenamiento más robusto para parámetros pocos frecuentes o con escalas distintas. Si un parámetro recibió grandes gradientes muchas veces, ADAGrad le baja el learning rate para que no haga pasos grandes y haga pasos más chicos. Si un parámetro recibió gradientes chicos muchas veces, ADAGrad le sube el learning rate.

Inciso (c)

Es para medir la magnitud del gradiente, independientemente del signo.

Inciso (d)

ADAM usa una suma de gradientes sin elevarlos al cuadrado para que no se pierda esa información del signo. La ventaja que tiene por sobre ADAGrad es que es más rápido, nada más.

Ejercicio 5

- **Falso.** No es convexa la función de pérdida. Tiene mínimos locales, puntos de silla de montar, llanuras, etcétera.
- **Falso.** No está garantizado encontrar el mínimo global, porque se puede caer en mínimos locales.
- **Verdadero.**
- **Falso.** El problema sigue siendo no convexo en todo su dominio, por lo que puede no haber un mínimo global.

Ejercicio 6

- **Verdadero.**
- **Falso.** Es al revés: está pensado para que acierte lo más posible en test, no en training.
- **Falso.** No agrega más pesos, sino que agrega un término de penalización L2 a la función de pérdida.
- **Verdadero.** Se debe considerar el parámetro λ que se usa como penalización.

Ejercicio 7

- **Verdadero.** Mejora la performance en test y empeora la performance en train.
- **Falso.** No modifica la cantidad de parámetros, sino que en una iteración puede usar un subconjunto y en otra iteración puede usar otros.
- **Verdadero.** Se usa la tasa de dropout, que asigna una proporción de neuronas para apagar durante cada iteración en el entrenamiento.
- **Verdadero.** Es una técnica general de regularización que no se limita al tipo de capa o tipo de neurona (densa o convolucional).
- **Verdadero.** No hay una restricción para usarlos en conjunto.